Problem A. Whirlpool

Problem Setter: Bholanath Das Niloy
Tester: Irfanur Rahman Rafio
Alt Writer: Irfanur Rahman Rafio
Category: Geometry
Tags: Linear Transformations, Complex Numbers, Sorting

The center can be any point in the plane, so a direct search is difficult. We can make the choice easier by looking at how the center affects the transformation. First, expand the two equations:

$$x' = ax - by + (1-a)c_x + bc_y,$$
$$y' = bx + ay - bc_x + (1-a)c_y.$$

The terms $ax - by$ and $bx + ay$ depend only on the starting point and the fixed values a and b. The center adds the same translation (d_x, d_y) to every transformed point, where

$$d_x = (1-a)c_x + bc_y, \qquad d_y = -bc_x + (1-a)c_y.$$

This gives us a simpler choice: find the translation that saves the most value. To use this idea, we must check that every translation is possible. The system above has determinant $(1-a)^2 + b^2$, which is positive because $b \neq 0$. Thus every translation corresponds to exactly one valid center.

Now suppose we want the treasure at spot i to land at protected spot j. The translation must make up the difference between the transformed source and this destination:

$$d_x = x_j - ax_i + by_i, \qquad d_y = y_j - bx_i - ay_i.$$

Each pair (i, j) therefore gives us a record (d_x, d_y, w_i). We use w_i because that is the value of the treasure we save.

The next step is to group records with the same translation. All pairs in a group work with the same center. Once we fix a translation, each treasure has only one destination. Since the protected spots are distinct, each source appears at most once in a group. The sum of the values in that group is therefore exactly the value its center saves.

We can now take the largest group sum. Every group gives a valid center, and every center that saves a treasure appears in one of the groups.

For example, in the third sample, $(2, 1) \to (2, 2)$ and $(2, 2) \to (1, 3)$ both need the translation $(1, -1)$. Their records go into the same group, so their values add up to $5 + 4 = 9$.

Algorithm 1: Maximum safe treasure value

```
R ← [ ]                                              ▷ Collect the required translations
for i ← 1 to n do
    for j ← 1 to n do
        dx ← x[j] − a · x[i] + b · y[i]
        dy ← y[j] − b · x[i] − a · y[i]
        Append (dx, dy, w[i]) to R
    end
end
Sort R by (dx, dy)                                   ▷ Equal translations are adjacent
answer ← 0
foreach group G with equal (dx, dy) in R do
    total ← the sum of the values in G
    answer ← max(answer, total)
end
return answer
```

The list contains n^2 records, including the pairs with $i = j$. Sorting takes $O(n^2 \log n)$ time, and the group scan takes $O(n^2)$ time. The space requirement is $O(n^2)$.

Use signed 64-bit integers throughout, including for the multiplications. Translation coordinates can reach $2 \cdot 10^{12} + 10^8$ in magnitude, and the answer can reach 10^{12}.

Problem B. The Rubber Stamp

Problem Setter: Muhiminul Islam Osim
Tester: Raihat Zaman Neloy
Category: Dynamic Programming, Strings
Tags: Suffix Automaton, Z-Function, Segment Tree

Problem Summary

Given a strip of length n and a stamp word P of length k, find the minimum number of stamp operations to produce target string T, or output -1 if impossible. Stamps may overhang either end of the strip.

Key Observations

1. **Contiguous Block Partition:** In the final strip, each cell is "owned" by the stamp that printed it last. A stamp cannot own two disconnected segments, as any intervening stamp would have length k, making the span $> k$. Thus, T partitions into contiguous blocks from left to right, where each block is a substring of P.
2. **Boundary Overhang Condition:** Between adjacent blocks B_1 and B_2, stamps cannot mutually overhang into each other (as that would require stamp 1 to be placed before stamp 2 and vice versa). Thus, at every cut, at least one stamp must not overhang:
   - Either block 1 ends flush (it is a **suffix** of P), or
   - Block 2 starts flush (it is a **prefix** of P).
3. **Acyclicity:** Overhang dependencies are strictly directional and cannot form cycles of length ≥ 2. Hence, this local boundary condition is necessary and sufficient for a valid stamping sequence to exist.

Dynamic Programming

We define two DP states for cut position $i \in [0, n]$:

- $\texttt{endsFlush}[i]$: Minimum stamps covering $T[1..i]$ where the last stamp ends flush at i.
- $\texttt{anyEnd}[i]$: Minimum stamps covering $T[1..i]$ with arbitrary right end for the last stamp.

Base case: $\texttt{endsFlush}[0] = 0$, $\texttt{anyEnd}[0] = \infty$.

For each position $r \in [1, n]$:

- **Block ending at r is a suffix of P (sufLen$[r]$):** Transitions from any flush cut $j \in [r - \text{sufLen}[r], r-1]$. If $\text{sufLen}[r] = k$ (entire word P), it can also transition from $\texttt{anyEnd}[r-k]$:

$$\texttt{endsFlush}[r] = 1 + \min\left(\min_{j \in [r-\text{sufLen}[r], r-1]} \texttt{endsFlush}[j], \quad [\text{sufLen}[r] = k] \cdot \texttt{anyEnd}[r-k]\right)$$

- **Arbitrary block ending at r (subLen$[r]$) or prefix block starting at l (prefLen$[l]$):**

$$\texttt{anyEnd}[r] = 1 + \min\Big(\min_{j \in [r-\text{subLen}[r], r-1]} \texttt{endsFlush}[j], \min_{\substack{l \leq r \\ l + \text{prefLen}[l] - 1 \geq r}} \min(\texttt{anyEnd}[l-1], \texttt{endsFlush}[l-1])\Big)$$

The answer for $T[1..n]$ is $\min(\texttt{endsFlush}[n], \texttt{anyEnd}[n])$ (or -1 if ∞).

Common Pitfall: Naive Substring Partition

A common trap is naive substring partitioning (minimum substrings of P concatenating to T), which only provides a lower bound and fails to detect cyclic dependencies. For example, with $T =$ `"aabbaa"` and $P =$ `"baba"`, a naive partition finds 4 blocks (`"a"` + `"ab"` + `"ba"` + `"a"`), but `"ab"` and `"ba"` mutually overhang into each other, creating an unresolvable cycle $2 \leftrightarrow 3$. The boundary flushness requirement correctly rules out this case and yields -1.

Implementation & Complexity

- prefLen$[l]$ and sufLen$[r]$ are computed in $O(n+k)$ using the Z-function on P+`#`+T and rev(P)+`#`+rev(T).
- subLen$[r]$ is computed in $O(n+k)$ via the Suffix Automaton of P.
- Range minimums on `endsFlush` are queried in $O(\log n)$ using a Segment Tree; active prefix blocks are maintained with a Min-Heap with lazy deletion in $O(\log n)$ amortized time.
- **Total Time:** $O\big(\sum(n \log n + k)\big)$.
- **Total Space:** $O(n+k)$.

Problem C. In Agreement

Problem Setter: Kawchar Husain
Tester: Raihat Zaman Neloy
Alt Writer: Sharif Minhazul Islam Emon
Category: Trees, Number Theory
Tags: GCD, Prefix Sum

Let $W = \sum_{i=1}^{n} a_i$ be the sum of all vertex weights.

Suppose we choose vertex r as the root. Since its subtree contains the entire tree, $s_r = W$. Therefore, all subtree sums must have the same remainder as W modulo k.

Condition for a Fixed Root

Consider any non-root vertex v. It has exactly $\deg(v) - 1$ children, where $\deg(v)$ is its degree in the original tree.

The subtree sum of v is

$$s_v = a_v + \sum_{u \text{ is a child of } v} s_u.$$

If every child subtree sum is congruent to W modulo k, then

$$s_v \equiv a_v + (\deg(v) - 1)W \pmod{k}.$$

For s_v to also be congruent to W, we need

$$a_v + (\deg(v) - 2)W \equiv 0 \pmod{k}.$$

Define $b_v = a_v + (\deg(v) - 2)W$.

If a choice of root r and k is valid, the above condition must hold for every non-root vertex v.

Conversely, suppose k divides b_v for every $v \neq r$. Starting from the leaves and moving upward, we can prove that every non-root subtree sum is congruent to W modulo k. Indeed, whenever the condition holds for all children of v, the equations above show that it also holds for v. The root already has subtree sum W.

Therefore, for a fixed root r, the choice of k is valid if and only if k divides every b_v with $v \neq r$.

Notice that b_v depends only on the original tree, not on the chosen root.

Finding the Best Root

For a fixed root r, the maximum valid value of k is

$$g_r = \gcd(|b_v| : v \neq r).$$

This gcd is positive: every tree with at least two vertices has at least two leaves, and each leaf v satisfies $b_v = a_v - W < 0$. After excluding any one vertex, at least one leaf remains.

Thus, we need to find the maximum gcd obtainable by removing exactly one element from $b_1, b_2, \ldots, b_n$.

We can do this efficiently using prefix and suffix gcd arrays.

Let $pref_i$ be the gcd of $|b_1|, \ldots, |b_i|$, and let suf_i be the gcd of $|b_i|, \ldots, |b_n|$.

For convenience, define $pref_0 = suf_{n+1} = 0$.

If vertex r is chosen as the root, then

$$g_r = \gcd(pref_{r-1}, suf_{r+1}).$$

After computing the prefix and suffix gcd arrays, we can evaluate g_r for every possible root in constant time per vertex.

The answer is the maximum value of g_r over all vertices.

Complexity Analysis

Computing W, the degrees, and the values b_v takes $O(n)$ time. Building the prefix and suffix gcd arrays and checking all possible roots requires $O(n)$ gcd operations.

Thus, the time complexity is $O(n \log M)$, where $M = \max_v |b_v|$, and the additional space complexity is $O(n)$.

Use 64-bit integers, since the values b_v can exceed the 32-bit integer range.

Problem D. Stuck in the Middle

Problem Setter: Kawchar Husain
Tester: Raihat Zaman Neloy
Alt Writer: MD. Shibly Rahman Alve, Irfanur Rahman Rafio
Category: Strings
Tags: Run-Length Encoding

Split the string into maximal blocks of consecutive equal characters. For example, `aaabbbbcc` has three blocks with lengths $3, 4, 2$.

Every odd-length substring contained in a single block is centered.

Now consider an odd-length substring intersecting exactly two blocks. Suppose it contains x characters from the first block and y characters from the second. Since $x + y$ is odd, $x \neq y$. The middle character belongs to the larger part, which contains more than half of the substring. Therefore, every such substring is centered.

Hence, we only need to consider substrings intersecting at least three blocks.

Three Consecutive Blocks

Consider three consecutive blocks with lengths a, b, c, and a substring that starts in the first block and ends in the third block. Such a substring contains the entire middle block. Suppose it contains x characters from the first block and y from the third, where $1 \le x \le a$ and $1 \le y \le c$.

First, suppose $b \ge a + c$.

Then $b \ge x + y$. Since the substring has odd length, $x + b + y$ is odd, so b and $x + y$ have different parity. Therefore, equality is impossible, and we must have $b > x + y$.

Thus, the middle position lies inside the middle block, and its character appears exactly b times, which is more than half of the substring. Therefore, every such odd-length substring is centered.

Now suppose $b < a + c$.

Since $2 \le b + 1 \le a + c$, we can choose integers x and y such that $1 \le x \le a$, $1 \le y \le c$, and $x + y = b + 1$.

Consider the substring consisting of these x characters, the entire middle block, and these y characters. Its length is $x + b + y = 2b + 1$.

Since $x, y \ge 1$ and $x + y = b + 1$, both x and y are at most b. Therefore, the middle position lies inside the middle block.

However, its character appears only b times, which is not more than half of the substring. Hence, this substring is not centered.

Therefore, all odd-length substrings spanning these three blocks are centered if and only if

$$b \ge a + c.$$

Four or More Blocks

Suppose the string contains at least four blocks. Consider any four consecutive blocks with lengths a, b, c, d.

If every odd-length substring were centered, the condition for the first three blocks would require $b \ge a + c$. Since $a \ge 1$, this implies $b > c$.

Similarly, the condition for the last three blocks would require $c \ge b + d$. Since $d \ge 1$, this implies $c > b$.

These two inequalities contradict each other.

Therefore, every string with at least four blocks contains an odd-length substring that is not centered.

Combining these observations, the answer is `YES` if the string has at most two blocks. If it has exactly three blocks with lengths a, b, c, the answer is `YES` if and only if $b \ge a + c$. For four or more blocks, the answer is always `NO`.

Complexity Analysis

Finding the maximal blocks takes $O(n)$ time. Storing their lengths takes $O(n)$ additional space.

Problem E. Perfect Couple

Problem Setter: Irfanur Rahman Rafio, Abdullah Abrar
Tester: Sharif Minhazul Islam Emon
Alt Writer: Raihat Zaman Neloy
Category: Number Theory, Range Query, Brute Force
Tags: Pigeonhole Principle

Start by factorizing the function: $f(x, y) = x^3y - xy^3 = xy(x^2 - y^2) = xy(x + y)(x - y)$.

You only need to know whether $f(x, y)$ is divisible by m, and that only depends on the **remainders** of x and y when divided by m. So, before calculating f, replace x and y by their remainders.

The factorization immediately shows a few easy ways to get an m-perfect couple:

- If x is divisible by m (its remainder is 0), then so is $f(x, y)$, whatever y is.
- If x and y have the same remainder, then $x - y$ is divisible by m.
- If the remainders of x and y add up to m, then $x + y$ is divisible by m. Call two such remainders **complements** of each other. For example, for $m = 7$, the complement of 2 is 5.

The most straightforward solution is to check every pair of indices in the subarray. But a subarray can have up to 10^6 elements, so that is too slow. It is intuitive that a long subarray should contain a couple anyway. But how long does it have to be?

To find out, play the adversary: try to build a long subarray without any couple. Look at the first query of the first sample. The values $16, 15, 11$ leave the remainders $2, 1, 4$ when divided by 7, and no two of them form a 7-perfect couple. Can you add one more number?

- A number with remainder 0 forms a couple with everything.
- A number with remainder 1, 2 or 4 repeats a remainder.
- A number with remainder 6, 5 or 3 is the complement of 1, 2 or 4.

So every remainder fails, and 3 numbers is the best you can do for $m = 7$. That's exactly what happens in the second query: it adds 9, whose remainder 2 is the same as the remainder of 16, and the answer is the pair of indices 1 and 4.

The same thing happens for any m. Pair up every non-zero remainder with its complement: 1 with $m - 1$, 2 with $m - 2$, and so on. Call each such pair a **class**. For $m = 7$, the classes are $\{1, 6\}$, $\{2, 5\}$ and $\{3, 4\}$. For $m = 8$, they are $\{1, 7\}$, $\{2, 6\}$, $\{3, 5\}$ and $\{4\}$ (the complement of 4 is 4 itself). Either way, there are exactly $\lfloor \frac{m}{2} \rfloor$ classes.

A subarray without a couple can't contain a number with remainder 0, and it can contain at most one number from each class. So it has at most $\lfloor \frac{m}{2} \rfloor$ numbers.

Claim: Any $\lfloor \frac{m}{2} \rfloor + 1$ numbers always contain an m-perfect couple.

Take any $\lfloor \frac{m}{2} \rfloor + 1 \geq 2$ numbers. If one of them has remainder 0, it forms a couple with any other one. Otherwise, all of them belong to the $\lfloor \frac{m}{2} \rfloor$ classes, so by the **pigeonhole principle**, at least two of them belong to the same class. These two have the same remainder or complementary remainders, so they form a couple. □

Notice that the claim only uses the easy couples. When m is not a prime, there are even more couples. For example, in the fourth query of the first sample, 9 and 4 have neither the same nor complementary remainders when divided by 10. But 9×4 gives a factor of 2 and $9 - 4$ gives a factor of 5, so $f(9, 4) = 2340$ is divisible by 10. Such couples only make it easier to find one, so they don't break the claim.

Now, the solution is simple. If the subarray $[l, r]$ is longer than $\lfloor \frac{m}{2} \rfloor + 1$, **trim** it to its first $\lfloor \frac{m}{2} \rfloor + 1$ elements. By the claim, the trimmed subarray still contains a couple, so you don't lose anything. Then check all the pairs of the trimmed subarray by brute force. If none of them is a couple, the answer is -1. And if there are several couples, as in the third query of the first sample, you can output any of them.

The trimmed subarray has at most 51 elements, so a query takes $\mathcal{O}(m^2)$ time. □

Notice that the claim works for **any** $\lfloor \frac{m}{2} \rfloor + 1$ elements of the subarray, not just the first ones. You could trim

to the last ones, or even to randomly chosen ones.

In fact, you don't even need to trim if you search in the right order. But the order matters a lot. Compare these two versions of the search, called on the whole subarray $[l, r]$ (with `a` as a global array):

Searching among the unseen elements	Searching among the seen elements

`pair<int, int> getCouple(int l, int r, int m)`
`{`
`  int i, j;`
`  for(i = l; i <= r; i++)`
`  for(j = i + 1; j <= r; j++)`
`  if(isPerfect(a[i], a[j], m))`
`  return {i, j};`
 	
`  return {-1, -1};`
`}`
	`pair<int, int> getCouple(int l, int r, int m)`
	`{`
	`  int i, j;`
	`  for(i = l; i <= r; i++)`
	`  for(j = l; j < i; j++)`
	`  if(isPerfect(a[i], a[j], m))`
	`  return {i, j};`
	
	`  return {-1, -1};`
	`}`

$\mathcal{O}(m \cdot (r-l))$ per query: TLE	$\mathcal{O}(m^2)$ per query

Both functions `return` as soon as they find a couple, without checking any more pairs. So the running time depends on how many pairs each one checks before it finds the first couple.

The left version is the usual way to check all the pairs. It fixes i and looks for its partner among all the elements after it, which it hasn't seen yet. It only returns at the first i that has a partner somewhere in the subarray. But the elements before it might have no partner at all, and for each of them, the inner loop scans the whole subarray. For example, take $m = 97$ and a subarray that starts with 47 elements from different classes, followed by a huge number of elements with the same remainder. The first 47 elements have no partner anywhere, so the left version scans the whole subarray 47 times.

The right version takes the elements one by one, and looks for a partner of each new element among the elements it has already seen. By the claim, the first $\lfloor \frac{m}{2} \rfloor + 1$ elements already contain a couple, so it returns before i goes beyond them. So it is the same as trimming, in disguise.

Time complexity per query $= \mathcal{O}(m^2)$.

Summary:

- $f(x, y) = xy(x+y)(x-y)$, so a remainder of 0, two equal remainders, or two complementary remainders always make a couple.
- Pairing every remainder with its complement and using the pigeonhole principle, it is guaranteed that any $\lfloor \frac{m}{2} \rfloor + 1$ elements contain a couple.
- If the given subarray is large, simply trim it to $\lfloor \frac{m}{2} \rfloor + 1$ elements and check all the pairs, in $\mathcal{O}(m^2)$ time per query.

Problem F. Yet Another Stone Problem

Problem Setter: MD. Shibly Rahman Alve
Tester: Muhiminul Islam Osim
Alt Writer: Kawchar Husain, Irfanur Rahman Rafio
Category: Game Theory, Constructive, Greedy
Tags: Game of Nim, Bitmasks

Overview

In this problem nobody plays the game. Instead, we must *design the starting position*: N positive piles adding up to S on which the named player wins under optimal play, or report that no such position exists.

We cannot build a winning position before we know what one looks like. So the first step is to understand the game itself and find a rule that decides the winner of any given position.

The game

A move chooses between 1 and K non-empty piles and strictly decreases each of them. For $K = 1$ this is ordinary Nim, whose losing positions are exactly those with XOR equal to 0. For general K we again look at the binary representations column by column.

Set $M = K + 1$. For a bit position b, let c_b be the number of piles that have bit b set.

Definition. A position is *balanced* if $c_b \equiv 0 \pmod{M}$ for every bit b.

For $M = 2$, "every c_b is even" is exactly "XOR = 0", so ordinary Nim is a special case.

Theorem (Moore's Nim). *The player to move loses if and only if the position is balanced.*

Proof. The empty position is balanced and losing, so by induction it suffices to show two facts.

(a) A move from a balanced position leads to an unbalanced one. Look at the highest bit b changed by the move. Piles only decrease, so every pile changing at bit b goes $1 \to 0$. If f piles do so, then $1 \le f \le K < M$, so c_b drops by f and is no longer divisible by M.

(b) From an unbalanced position some move reaches a balanced one. Let b be the highest bit with $c_b \not\equiv 0$, and $r = c_b \bmod M \le K$. Clear bit b in r piles that have it. These piles have now decreased, so their lower bits can be set freely. Go through the lower bits from high to low. If the untouched piles have u set bits in the current column and $s = u \bmod M \neq 0$, either set the bit in $M - s$ of the free piles, or, if there are fewer than $M - s$ of them, free s more piles by clearing this bit in s piles that have it (the number of chosen piles stays $\le M - 1 = K$). Every column now has count $\equiv 0 \pmod{M}$.

For our problem the theorem gives the targets directly:

$$\text{Bob wins} \iff \text{the starting position is balanced}, \qquad \text{Alice wins} \iff \text{it is not balanced.}$$

We can now decide the winner of any position, so we turn to the actual task: finding piles with the right winner *and* the right total. The winning condition depends only on the numbers c_b, and so does the total:

$$S = \sum_{i=1}^{N} A_i = \sum_{b \ge 0} c_b \cdot 2^b. \tag{1}$$

So we first choose a vector (c_b) with the right sum and balance, and only then spread its bits over the N piles. Since piles must be positive, not every vector can be spread.

Which vectors can be spread over N positive piles?

Lemma 1. *A vector (c_b) is the column-count vector of some N positive integers if and only if*

$$0 \le c_b \le N \ \text{ for all } b \qquad \text{and} \qquad \sum_b c_b \ \ge \ N.$$

Proof. Necessity. A bit is set in at most N piles, and each positive pile has at least one set bit.

Sufficiency. Walk around the N piles cyclically: for bit 0 set it in the next c_0 piles, for bit 1 in the next c_1 piles, and so on. Since $c_b \le N$, a bit never lands twice on one pile, and since the walk makes $\sum c_b \ge N$ steps, it visits every pile, so every pile gets a bit. By (1) the sum is S.

So we need a vector (c_b) with

$$\sum_b c_b 2^b = S, \qquad 0 \le c_b \le N, \qquad \sum_b c_b \ge N,$$

and the balance property the target requires. The condition $\sum_b c_b \geq N$ asks for *many* set bits, while the binary form of a number has *few*. So we need to write a number with as many digits as the cap allows.

A greedy for the largest digit sum

Given $X \geq 0$ and a cap $D \geq 1$, we want to write $X = \sum_b d_b 2^b$ with $0 \leq d_b \leq D$ and the digit sum $\sum_b d_b$ as large as possible. Choosing the counts c_b is exactly a problem of this form, once we decide what plays the role of the digits:

- **Alice.** The counts themselves are the digits: we need $\sum_b c_b 2^b = S$ with $c_b \leq N$. So $X = S$ and $D = N$.
- **Bob.** Every count must be a multiple of M, so write $c_b = M\, d_b$ and let the d_b be the digits. Dividing $\sum_b c_b 2^b = S$ by M gives $\sum_b d_b 2^b = S/M$, and $c_b \leq N$ gives $d_b \leq \lfloor N/M \rfloor$. So $X = S/M$ and $D = \lfloor N/M \rfloor$.

In both cases we need the digit sum to be *at least* some bound, so it is enough to find the largest possible digit sum: if even that falls short, no answer exists.

The greedy $\mathrm{maxRep}(X, D)$**.** Go from bit 0 upwards, with X being the value still to represent, in units of 2^b. The digit d_b must have the parity of X, since all higher bits contribute even multiples of 2^b. Take d_b as the largest value $\leq \min(D, X)$ with that parity, replace X by $(X - d_b)/2$, and stop when $X = 0$. Since X at least halves each step, this takes about 60 steps.

Lemma 2. $\mathrm{maxRep}(X, D)$ *maximizes* $\sum_b d_b$ *among all representations of* X *with digits in* $[0, D]$.

Proof. Let d_0^* be the greedy choice. Any valid d_0 is d_0^* or at most $d_0^* - 2$ (same parity). Suppose an optimal representation has $d_0 \leq d_0^* - 2$. Then some higher digit is positive; take the lowest, at bit $j \geq 1$, and split one unit of it into two units at bit $j - 1$. The value is unchanged and the digit sum grows by 1. The split is legal: if $j = 1$ then $d_0 + 2 \leq d_0^* \leq D$, and if $j > 1$ the new digit is $2 \leq D$ because $D \geq d_0^* \geq 2$. This contradicts optimality, so $d_0 = d_0^*$, and the rest is the same problem for $(X - d_0^*)/2$.

Intuitively, the greedy pushes as much value as possible into the low bits, where it yields the most digits. We begin with Bob, because the balance condition is the more restrictive one.

Target: Bob

We need a balanced position, so $c_b = M\, d_b$ for non-negative integers d_b. The conditions of Lemma 1 become:

- $c_b \leq N$ becomes $d_b \leq D := \lfloor N/M \rfloor$.
- $\sum c_b \geq N$ becomes $\sum d_b \geq R := \lceil N/M \rceil$.
- By (1), M must divide S, and then $\sum_b d_b 2^b = V := S/M$.

Algorithm.

1. If $M > N$, print -1: then $D = 0$, so $V = 0$, contradicting $S \geq 1$. (Alice can empty all piles in one move.)
2. If $S \bmod M \neq 0$, print -1.
3. Compute $d = \mathrm{maxRep}(V, D)$. If $\sum d_b < R$, print -1.
4. Otherwise set $c_b = M\, d_b$ and build the piles as in Lemma 1.

Correctness. Any valid answer gives such a vector (d_b), and by Lemma 2 none has a larger digit sum than the greedy one. So if the greedy falls below R, the answer is -1. Otherwise $c_b = M d_b \leq N$ and $\sum c_b \geq MR \geq N$, so Lemma 1 applies.

For Alice, we only need *some* column to be off, so the difficulty lies in the few cases where the sum leaves almost no freedom.

Target: Alice

We need a vector with $\sum c_b 2^b = S$, $0 \le c_b \le N$, $\sum c_b \ge N$, and at least one $c_b \not\equiv 0 \pmod{M}$.

Case $S < N$. Positive piles cannot sum to less than N. Print -1.

Case $S = N$. The only array is $(1, 1, \ldots, 1)$, which has $c_0 = N$. It is a win for Alice if and only if $M \nmid N$. If $M \mid N$, print -1.

Case $S > N$. Let $c = \text{maxRep}(S, N)$. If some $c_b \not\equiv 0 \pmod{M}$, we are done, since c is valid and unbalanced (justified below). Otherwise c is balanced and we perform one carry: let b be the highest non-zero bit of c, and set

$$c_b \leftarrow c_b - 2, \qquad c_{b+1} \leftarrow c_{b+1} + 1.$$

Why this works. Since $S > N$, the greedy gives $c_0 \in \{N, N-1\}$ and the remaining value is positive. So $\sum c \ge N$ (the vector is valid), with equality only if $c_0 = N - 1$ and the rest is a single unit $c_j = 1$. Such a vector has an entry equal to 1, so it is unbalanced. Hence a balanced c has $\sum c \ge N + 1$.

For the carry: the non-zero entries of a balanced vector are multiples of $M \ge 2$, so $c_b \ge 2$, and $c_{b+1} = 0$ because b is the highest non-zero bit. After the carry, $c_{b+1} = 1 \le N$ is not divisible by M, so the vector is unbalanced, and $\sum c$ dropped by only 1, so it is still $\ge N$.

Hence for $S > N$ the answer always exists, and the construction above produces it.

Complexity. $O(\sum N \log S)$ time and $O(N)$ memory.

Problem G. Optimal Deployment

Problem Setter: Foyaz Akanda
Tester: Raihat Zaman Neloy
Category: Dynamic Programming, Trees
Tags: Rerooting DP, Suffix Minimum

Precomputing Server Costs

Each operation requires the cheapest server with capacity at least some x. Since any subtree has at most N modules and any node has at most $N - 1$ children, we only ever query $0 \le x \le N$, and capacities larger than N can be clamped to N.

Let $\texttt{best}[x]$ be the minimum cost among all servers with capacity at least x (or ∞ if none exists). We can precompute $\texttt{best}[0 \ldots N]$ in $O(N + M)$ time:

1. Initialize $\texttt{best}[0 \ldots N] = \infty$.
2. For each server i, set $c = \min(C_i, N)$ and update $\texttt{best}[c] = \min(\texttt{best}[c], P_i)$.
3. Compute suffix minimums from $x = N - 1$ down to 0:

$$\texttt{best}[x] = \min(\texttt{best}[x], \texttt{best}[x + 1]).$$

DP for a Fixed Root

For a fixed root, let $\texttt{sz}[u]$ be the subtree size of node u, k be its number of children, and $\texttt{in}[u]$ be the minimum cost to deploy its subtree. We have two choices at u:

1. **Deploy the entire subtree of** u in one operation at cost $\texttt{best}[\texttt{sz}[u]]$.
2. **Deploy only** u at cost $\texttt{best}[k]$, and deploy each child's subtree separately.

Thus, we can compute $\texttt{in}[u]$ for all nodes in $O(N)$ time via a bottom-up DFS:

$$\texttt{in}[u] = \min\Big(\texttt{best}[\texttt{sz}[u]],\ \texttt{best}[k] + \sum_{v \in \text{children}(u)} \texttt{in}[v]\Big).$$

Running this DP from every possible entry point takes $O(N^2)$ time. Instead, we use rerooting DP to evaluate all entry points in $O(N)$ time.

Rerooting DP

Root the tree arbitrarily at node 1 and compute $\texttt{sz}[u]$ and $\texttt{in}[u]$ bottom-up. Next, define $\texttt{out}[u]$ as the minimum cost to deploy the component outside u's subtree (of size $N - \texttt{sz}[u]$, rooted at u's parent), with $\texttt{out}[1] = 0$.

When u is chosen as the entry point, all $\deg(u)$ of its neighbors become its children: each original child v has branch cost $\texttt{in}[v]$, and u's parent (if $u \neq 1$) has branch cost $\texttt{out}[u]$. Let $\texttt{total}[u]$ be the sum of branch costs over all neighbors of u:

$$\texttt{total}[u] = \texttt{out}[u] + \sum_{v \in \text{children}(u)} \texttt{in}[v].$$

We compute $\texttt{ans}[u]$ and $\texttt{out}[v]$ in a single top-down DFS (where $\texttt{out}[u]$ is already known when visiting u) by applying the same two deployment choices at u:

- **Evaluate u as the entry point:** The full tree has size N, and u has $\deg(u)$ children with total branch cost $\texttt{total}[u]$:
$$\texttt{ans}[u] = \min\big(\texttt{best}[N],\ \texttt{best}[\deg(u)] + \texttt{total}[u]\big).$$
- **Transition to each child v of u:** Removing v's subtree leaves a component rooted at u of size $N - \texttt{sz}[v]$, where u has $\deg(u) - 1$ children (all neighbors except v):
$$\texttt{out}[v] = \min\big(\texttt{best}[N - \texttt{sz}[v]],\ \texttt{best}[\deg(u) - 1] + \texttt{total}[u] - \texttt{in}[v]\big).$$

The final answer is $\min_{1 \le u \le N} \texttt{ans}[u]$, or -1 if this minimum is ∞.

Implementation Notes:

- **Handling ∞:** Since branch costs can be ∞, avoid direct subtraction in $\texttt{total}[u] - \texttt{in}[v]$. Instead, maintain the sum of finite neighbor costs and the count of ∞ neighbors at u (or use prefix and suffix sums over u's neighbors).
- **64-bit Integers:** Total cost can reach $N \cdot \max P_i = 10^{14}$, so use 64-bit integers and set $\infty > 10^{14}$ (e.g., 10^{18}).

Complexity Analysis

- **Time Complexity:** $O(N + M)$ per test case, and $O\big(\sum(N + M)\big)$ overall.
- **Space Complexity:** $O(N + M)$.

Problem H. Vanishing Fairy Tale

Problem Setter: Sharif Minhazul Islam
Tester: Raihat Zaman Neloy
Category: Combinatorics, Divide and Conquer
Tags: FFT/NTT, Inclusion-Exclusion Principle

Let g_d be the number of spells with beauty exactly d. Once all g_d are known, the answer for K is $\sum_{K|d} g_d$. Beauty never exceeds $2N + 1$, so we precompute this sum for every $K \le 2N + 1$ in $O(N \log N)$ (harmonic series). For larger K, only $d = 0$ counts, and the answer is $g_0 = 1$.

Groups. Group equal values. Suppose value v occurs C times. Its elements can only produce $+v$, $-v$ or 0. The integers $\pm v$ belong to this group alone, but 0 is shared by all groups. Let the *local beauty* of a group be how many of $+v, -v$ appear (0, 1 or 2). For one group, define two polynomials whose coefficient of x^d counts assignments of the C elements with local beauty d:

- $A(x)$: no element of the group vanishes (each element is skipped, kept or flipped);
- $T(x)$: no restriction (each element is skipped, kept, flipped or vanishes).

By inclusion–exclusion on "$+v$ is not used" and "$-v$ is not used":

$$A(x) = 1 + (2 \cdot 2^C - 2)\, x + (3^C - 2 \cdot 2^C + 1)\, x^2,$$

$$T(x) = 2^C + (2 \cdot 3^C - 2 \cdot 2^C)\, x + (4^C - 2 \cdot 3^C + 2^C)\, x^2.$$

Handling the shared zero. Split all spells by whether *anything* vanishes. If nothing vanishes, the groups are independent, and these spells are counted by $\prod A_i(x)$. All spells together are counted by $\prod T_i(x)$ (by local beauty only). So the spells where something vanishes are counted by $\prod T_i - \prod A_i$, and each of them gets exactly one extra distinct integer, 0. Therefore

$$G(x) = \sum_d g_d x^d = \prod_i A_i(x) + x\Big(\prod_i T_i(x) - \prod_i A_i(x)\Big) = (1-x)\prod_i A_i(x) + x\prod_i T_i(x).$$

Computing the products. There are $m \le N$ groups, so each product is a product of m quadratics, with degree $2m$. Multiply them by divide and conquer with NTT modulo 998244353. That is $O(N \log^2 N)$ per test case, and $O\left(\sum N \log^2 N\right)$ overall. The maximum length is $2 \cdot 10^5 + 1$, so the NTT size is 2^{18}, well within the 2^{23} limit of this modulus.

Alternative without NTT. Groups with the same size C have identical polynomials, and there are only $k = O(\sqrt{N})$ distinct sizes. Write $F = \prod_c P_c^{e_c}$, $L = \prod_c P_c$ and $M = \sum_c e_c P_c' \prod_{d \ne c} P_d$. Then $LF' = MF$, and $\deg L, \deg M \le 2k$. Comparing coefficients computes each coefficient of F from the previous $2k$ ones, which is $O(N\sqrt{N})$ in total. $L(0) \ne 0$ in both cases, so the division is valid. This also fits the time limit.

The naive $O(m^2)$ multiplication of the quadratics one by one is too slow.